Lab 40 — Automation and Orchestration with a Mini-SOAR Playbook

> **SY0-701 Objective 4.7** — Explain the importance of automation and orchestration related to secure operations.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~15 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: automate user provisioning from an HR feed and an SSH brute-force response playbook that blocks, opens a ticket and escalates through an API, with guard rails and CI-style tests.

You'll build: `provision.sh` (HR feed to accounts and security groups), `soar.sh` (detect, decide, act, record), a mock ticketing/paging API, a systemd service that runs the playbook continuously and a rollback command.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 38 (identity lifecycle) is useful background; tools installed in step 1.

Background

- **Use cases** on the exam: user and resource provisioning, guard rails, security groups, ticket creation, escalation, enabling or disabling services and access, continuous integration and testing, and integrations through APIs. This lab touches every one of them.
- **Benefits:** efficiency and time saving, enforcing baselines and standard configurations, scaling securely, faster reaction time, acting as a workforce multiplier, and employee retention because analysts stop doing repetitive toil.
- **Other considerations:** complexity, cost, the automation platform becoming a **single point of failure**, **technical debt** from quick scripts nobody maintains, and **ongoing supportability**.
- **SOAR** (security orchestration, automation and response) runs **playbooks**: machine-executable versions of the manual **runbooks** analysts follow. A SIEM detects; SOAR acts.
- **Guard rails** are automated limits that stop an automation, or a person, doing something unsafe, such as blocking your own management address.

Steps

1. Install the tools

`jq` builds JSON tickets, `shellcheck` is the linter for the CI step, and Python provides a mock REST API that stands in for a ticketing or paging platform.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y jq shellcheck
iptables curl python3
mkdir -p /etc/soar /var/soar/tickets
```

2. Automate user provisioning from an HR feed

HR is the source of truth. The script creates accounts for active staff, puts them in a department security group and disables leavers. It is idempotent, so running it twice changes nothing the second time, which is what makes automation safe to schedule.

```
cat > /etc/soar/hr_feed.csv <<'EOF'
username,department,status
dtan,finance,active
mlim,engineering,active
rkumar,engineering,leaver
EOF
id rkumar >/dev/null 2>&1 || useradd -m rkumar
cat > /usr/local/bin/provision.sh <<'EOF'
#!/usr/bin/env bash
# provision.sh - create active staff, add to department security group, disable leavers
set -euo pipefail
FEED="${1:-/etc/soar/hr_feed.csv}"
while IFS=, read -r user dept status; do
  [ "$user" = "username" ] && continue
  getent group "sg-$dept" >/dev/null || groupadd "sg-$dept"
  if [ "$status" = "active" ]; then
    if id "$user" >/dev/null 2>&1; then echo "OK $user already provisioned"
    else useradd -m -G "sg-$dept" -c "dept=$dept" "$user"; echo "PROVISIONED $user ->
sg-$dept"; fi
  elif id "$user" >/dev/null 2>&1; then
    usermod -L -e 1 "$user"; echo "DISABLED $user (leaver)"
  fi
done < "$FEED"
EOF
chmod +x /usr/local/bin/provision.sh
provision.sh
provision.sh
getent group sg-finance sg-engineering; passwd -S rkumar
```

3. Start a mock ticketing and paging API

Real playbooks call ServiceNow, Jira, Slack or PagerDuty through REST APIs. This small listener accepts JSON on `http://127.0.0.1:8088/hook` and records every escalation it receives.

```
cat > /usr/local/bin/mock-api.py <<'EOF'
import datetime, json
from http.server import BaseHTTPRequestHandler, HTTPServer
class Hook(BaseHTTPRequestHandler):
    def do_POST(self):
        body = self.rfile.read(int(self.headers.get("Content-Length", 0))).decode()
        with open("/var/soar/escalations.log", "a") as f:
            f.write(datetime.datetime.now(datetime.timezone.utc).isoformat() + " " + body
+ "\n")
        self.send_response(201)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps({"status": "on-call paged"}).encode())
    def log_message(self, *args):
        pass
```

```
HTTPServer(("127.0.0.1", 8088), Hook).serve_forever()
EOF
nohup python3 /usr/local/bin/mock-api.py >/dev/null 2>&1 &
sleep 1
curl -s -X POST -H 'Content-Type: application/json' -d '{"test":"ping"}'
http://127.0.0.1:8088/hook; echo
```

```
{"status": "on-call paged"}
```

4. Write the playbook with guard rails

The playbook reads log lines on standard input, so the same code runs against a test file in CI and against the live journal in production. Guard rails: an allow list of addresses that must never be blocked, a threshold, and a `DRY_RUN` mode.

```
printf '127.0.0.1\n10.0.0.5\n' > /etc/soar/allowlist
cat > /usr/local/bin/soar.sh <<'EOF'
#!/usr/bin/env bash
# soar.sh - mini SOAR playbook: SSH brute force -> block -> ticket -> escalate
set -u
THRESHOLD="${THRESHOLD:-5}"
DRY_RUN="${DRY_RUN:-0}"
ALLOWLIST="${ALLOWLIST:-/etc/soar/allowlist}"
TICKETS="${TICKETS:-/var/soar/tickets}"
API="${API:-http://127.0.0.1:8088/hook}"
mkdir -p "$TICKETS"
declare -A count
while IFS= read -r line; do
  case "$line" in
    *"Failed password"*|*"Invalid user"*) ;;
    *) continue ;;
  esac
  ip=$(grep -oE 'from [0-9]{1,3}(\.[0-9]{1,3}){3}' <<<"$line" | awk '{print $2}')
  [ -n "$ip" ] || continue
  count[$ip]=$(( ${count[$ip]:-0} + 1 ))
  [ "${count[$ip]}" -eq "$THRESHOLD" ] || continue
  if grep -qxF "$ip" "$ALLOWLIST" 2>/dev/null; then
    sev="low"; action="alert-only (guard rail: allowlisted)"
  else
    sev="high"; action="blocked"
    if [ "$DRY_RUN" = "1" ]; then
      action="would-block (dry run)"
    else
      iptables -C INPUT -s "$ip" -j DROP 2>/dev/null || iptables -I INPUT -s "$ip" -j DROP
    fi
  fi
  id="INC-$(date +%Y%m%d%H%M%S)-${ip//./-}"
  jq -n --arg id "$id" --arg ip "$ip" --arg sev "$sev" --arg act "$action" --argjson n
"${count[$ip]}" \
    '{id:$id, type:"ssh-brute-force", source_ip:$ip, failures:$n, severity:$sev,
action:$act, ts:(now|todate)}' \
    > "$TICKETS/$id.json"
  logger -t soar "ticket=$id ip=$ip severity=$sev action=$action"
  if [ "$sev" = "high" ] && [ "$DRY_RUN" != "1" ]; then
    curl -s -m 3 -X POST -H 'Content-Type: application/json' -d @"$TICKETS/$id.json"
```

```
"$API" >/dev/null \
      || logger -t soar "escalation API unreachable for $id"
  fi
  echo "$id $ip severity=$sev action=$action"
done
EOF
chmod +x /usr/local/bin/soar.sh
```

5. Test the playbook the CI way before it touches production

Continuous integration runs the same checks on every change: syntax, lint, then a dry run against known test data with an expected result. A playbook that has not been tested will fail at machine speed.

```
bash -n /usr/local/bin/soar.sh && echo "syntax OK"
shellcheck -S warning /usr/local/bin/soar.sh /usr/local/bin/provision.sh && echo "lint OK"
{ for i in 1 2 3 4 5; do echo "sshd[1]: Failed password for root from 198.51.100.9 port
4$i ssh2"; done
  for i in 1 2 3 4 5; do echo "sshd[1]: Failed password for admin from 127.0.0.1 port 5$i
ssh2"; done; } > /tmp/ci-sample.log
DRY_RUN=1 TICKETS=/tmp/ci-tickets /usr/local/bin/soar.sh < /tmp/ci-sample.log
iptables -C INPUT -s 198.51.100.9 -j DROP 2>/dev/null && echo "FAIL: dry run changed the
firewall" || echo "PASS: dry run made no changes"
```

```
INC-...-198-51-100-9 198.51.100.9 severity=high action=would-block (dry run)
INC-...-127-0-0-1 127.0.0.1 severity=low action=alert-only (guard rail: allowlisted)
PASS: dry run made no changes
```

6. Run the playbook as a service

Orchestration means the automation runs continuously and restarts itself. The service follows new `sshd` entries in the systemd journal and pipes them into the playbook.

```
cat > /etc/systemd/system/soar.service <<'EOF'
[Unit]
Description=Mini SOAR SSH brute-force playbook
After=network.target

[Service]
ExecStart=/bin/bash -c 'journalctl -f -n0 -o short -t sshd | /usr/local/bin/soar.sh'
Restart=always

[Install]
WantedBy=multi-user.target
EOF
systemctl daemon-reload && systemctl enable --now soar.service
systemctl is-active soar.service
```

7. Trigger an incident and watch the automated response

Simulate a brute-force burst from a documentation (TEST-NET) address by writing sshd-style events into the journal, then check every automated output: firewall block, ticket, log entry and API escalation.

```
for i in $(seq 1 6); do logger -p auth.warning -t sshd "Failed password for invalid user
admin from 203.0.113.50 port 50$i ssh2"; done
sleep 3
iptables -L INPUT -n | grep 203.0.113.50
ls /var/soar/tickets/ && jq . /var/soar/tickets/*203-0-113-50*.json
journalctl -t soar -n 3 --no-pager
cat /var/soar/escalations.log | tail -1
```

```
DROP       all  --  203.0.113.50         0.0.0.0/0
{ "id": "INC-...-203-0-113-50", "type": "ssh-brute-force", "source_ip": "203.0.113.50",
"failures": 5, "severity": "high", "action": "blocked", ... }
```

8. Roll back: re-enable access after review

Every automated action needs a documented, equally automated way back. After the analyst confirms the block (or finds a false positive), the ticket is closed and access is re-enabled.

```
iptables -D INPUT -s 203.0.113.50 -j DROP && echo "access re-enabled for 203.0.113.50"
T=$(ls /var/soar/tickets/*203-0-113-50*.json | head -1)
jq '.status="closed" | .resolution="reviewed by analyst, block lifted"' "$T" > "$T.tmp" &&
mv "$T.tmp" "$T"
jq -r '.id + " " + .status' "$T"
logger -t soar "rollback ip=203.0.113.50 ticket=$(jq -r .id "$T")"
```

Test it

```
systemctl is-active soar.service
for i in $(seq 1 5); do logger -p auth.warning -t sshd "Failed password for root from
198.51.100.77 port 61$i ssh2"; done; sleep 3
iptables -L INPUT -n | grep -c 198.51.100.77
grep -c 198.51.100.77 /var/soar/escalations.log
```

Expected: the service is `active`, and both counts are `1` — the new attacker was blocked once and escalated once, with no human involved.

Troubleshooting

Symptom	Fix
No ticket appears after step 7	Check the service with `journalctl -u soar -n 20`. If `journalctl -t sshd` shows nothing, the journal is not running; feed the playbook manually with `journalctl -t sshd -o short | soar.sh`.
Escalation log is empty	The mock API is not running (it does not survive a reboot). Re-run the `nohup python3 ...` line from step 3 and test it with the curl ping.
You locked yourself out by blocking the wrong address	This is why the allow list exists. Remove the rule with `iptables -D INPUT -s <ip> -j DROP` and add your management address to `/etc/soar/allowlist`.

Challenge

Add a second guard rail: the playbook may block at most 10 addresses per hour, and above that it must stop blocking, raise a critical ticket and escalate to a human (a runaway playbook is itself an incident). Add a CI test that feeds 20 attacker addresses and asserts only 10 are blocked.

Reflection

Your SOAR platform now blocks addresses, disables accounts and pages staff. Using the "other considerations" in objective 4.7, what happens on the day this one service fails or is misconfigured, and how do you reduce that risk?

Exam tips

- SIEM correlates and alerts; SOAR orchestrates and responds. "Automatically disable the account and open a ticket" points to SOAR.
- Runbook = documented procedure for people; playbook = the automated (or checklist) workflow that executes it.
- "Workforce multiplier" and "reaction time" are benefits; "single point of failure" and "technical debt" are considerations (drawbacks).
- Guard rails prevent unsafe actions; security groups apply access consistently to many resources at once.

References

- NIST SP 800-61 Computer Security Incident Handling Guide — https://csrc.nist.gov/pubs/sp/800/61/r3/final
- MITRE ATT&CK T1110 Brute Force — https://attack.mitre.org/techniques/T1110/
- Shuffle open-source SOAR — https://shuffler.io
- TheHive case management — https://strangebee.com/thehive/
- ShellCheck — https://www.shellcheck.net
